



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Introduction to Web Technology

Course code: 2344211

Year: SE SEM: IV

Experiment No. 5
Aim: Develop a web application to fetch real-time weather data for a city. Use AJAX & a public weather API to retrieve information. Display temperature, humidity, and weather conditions dynamically.
Name: Saniya Laxman Parab
Roll Number: 52
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Checked by

Name of Faculty :

Signature :

Date



Experiment 5

Title

Develop a web application to fetch real-time weather data for a city. Use AJAX & a public weather API to retrieve information. Display temperature, humidity, and weather conditions dynamically.

Theory:

A weather web application provides real-time climate information by communicating with external servers using web technologies. In this experiment, AJAX (Asynchronous JavaScript and XML) is used to fetch live weather data from a public Weather API without refreshing the webpage.

The Weather API returns data in JSON format, which contains various weather parameters such as temperature, humidity, and weather description. JavaScript processes this data and updates the webpage dynamically using DOM manipulation.

AJAX improves user experience by allowing asynchronous data transfer between client and server. The user enters a city name, and the application retrieves and displays updated weather information instantly.

Role of AJAX

AJAX (Asynchronous JavaScript and XML) enables web applications to send and receive data from a server **asynchronously**, without refreshing the webpage. This improves performance and user experience. In this experiment, AJAX is implemented using the **Fetch API**, which allows HTTP requests such as GET to retrieve data from the Weather API. Although the name includes XML, modern AJAX applications mostly exchange data in **JSON format**.

Fetch API

The Fetch API is a modern JavaScript interface used to make HTTP requests. It returns a **Promise**, which allows handling asynchronous operations efficiently. Once the API response is received, it is converted into JSON format and processed further. The Fetch API simplifies AJAX implementation compared to older methods like XMLHttpRequest.

Weather API and API Key

A Weather API is a web service that provides access to weather data such as temperature, humidity, pressure, and weather conditions. Most Weather APIs require an **API key**, which acts as a unique identifier to authenticate users and track API usage. The API key ensures secure access and prevents misuse of the service.



JSON Data Processing

JSON (JavaScript Object Notation) is a lightweight data-exchange format that is easy to read and write. The Weather API sends weather information in JSON format. JavaScript accesses this data using object notation to extract required values like temperature, humidity, and weather description.

DOM Manipulation

The **Document Object Model (DOM)** represents the structure of a webpage as a tree of objects. JavaScript uses DOM manipulation techniques to dynamically update webpage content. In this application, the fetched weather details are inserted into HTML elements using DOM methods, allowing real-time updates without page reload.

Dynamic Web Page Behavior

Dynamic behavior is achieved when the webpage content changes based on user interaction. In this experiment, when the user clicks the button after entering a city name, the application fetches fresh weather data and updates the display area instantly. This makes the application interactive and responsive.

Error Handling

Error handling is an important aspect of web applications. If the user enters an invalid city name or if there is a network issue, the application displays an appropriate error message. This ensures better usability and reliability.

Algorithm / Procedure

1. Start the experiment.
2. Create an HTML file for user interface.
3. Add an input field to enter the city name.
4. Add a button to fetch weather details.
5. Use JavaScript to capture the city name entered by the user.
6. Send an AJAX request to the Weather API using `fetch()`.
7. Receive weather data in JSON format.



8. Extract temperature, humidity, and weather condition from the response.
9. Display the fetched data dynamically on the webpage.
10. Stop the experiment.

Program / Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Weather App</title>
  <style>
    body {
      font-family: Arial;
      background: #eef2f3;
      text-align: center;
    }
    .box {
      background: white;
      padding: 20px;
      width: 300px;
      margin: auto;
      box-shadow: 0 0 10px gray;
    }
  </style>
</head>
<body>

<div class="box">
  <h2>Weather Application</h2>
  <input type="text" id="city" placeholder="Enter city name">
  <br><br>
  <button onclick="getWeather()">Get Weather</button>
  <p id="result"></p>
</div>

<script>
function getWeather() {
  var city = document.getElementById("city").value;
  var apiKey = "YOUR_API_KEY";
  var url = "https://api.openweathermap.org/data/2.5/weather?q="
```



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
+ city + "&units=metric&appid=" + apiKey;
```

```
fetch(url)
.then(response => response.json())
.then(data => {
    document.getElementById("result").innerHTML =
        "Temperature: " + data.main.temp + " °C<br>" +
        "Humidity: " + data.main.humidity + "%<br>" +
        "Condition: " + data.weather[0].description;
})
.catch(() => {
    document.getElementById("result").innerHTML = "City not found!";
});
}
</script>

</body>
</html>
```

CODE:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Weather App</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<style>
```

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

```
body {
```

```
    font-family: 'Segoe UI', sans-serif;
```

```
    height: 100vh;
```

```
    display: flex;
```

```
    justify-content: center;
```

```
    align-items: center;
```

```
    background: linear-gradient(135deg, #667eea, #764ba2);
```



```
        overflow: hidden;
    }

    /* Background floating animation */
    .background-animation {
        position: absolute;
        width: 200%;
        height: 200%;
        background: radial-gradient(circle at 20% 30%, rgba(255, 255, 255, 0.2), transparent 40%),
            radial-gradient(circle at 70% 70%, rgba(255, 255, 255, 0.2), transparent 40%);
        animation: float 10s infinite alternate ease-in-out;
    }

    @keyframes float {
        from {
            transform: translate(-10%, -10%) rotate(0deg);
        }

        to {
            transform: translate(10%, 10%) rotate(10deg);
        }
    }

    .container {
        position: relative;
        background: rgba(255, 255, 255, 0.15);
        padding: 40px;
        border-radius: 20px;
        backdrop-filter: blur(15px);
        box-shadow: 0 20px 40px rgba(0, 0, 0, 0.3);
        text-align: center;
        width: 350px;
        animation: fadeIn 1s ease-in-out;
    }

    @keyframes fadeIn {
        from {
            opacity: 0;
            transform: translateY(20px);
        }
    }
```



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
to {
  opacity: 1;
  transform: translateY(0);
}
}

h1 {
  margin-bottom: 20px;
  color: white;
  letter-spacing: 1px;
}

.input-group {
  display: flex;
  gap: 10px;
  justify-content: center;
}

input {
  padding: 12px;
  width: 200px;
  border: none;
  border-radius: 10px;
  outline: none;
  font-size: 14px;
  transition: 0.3s;
}

input:focus {
  box-shadow: 0 0 10px rgba(255, 255, 255, 0.7);
}

button {
  padding: 12px 18px;
  border: none;
  border-radius: 10px;
  background: #ff7eb3;
  color: white;
  font-weight: bold;
}
```



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
        cursor: pointer;
        transition: 0.3s;
    }

    button:hover {
        background: #ff4f9a;
        transform: scale(1.05);
    }

    .result-card {
        margin-top: 25px;
        padding: 20px;
        border-radius: 15px;
        background: rgba(255, 255, 255, 0.2);
        color: white;
        min-height: 60px;
        transition: 0.4s;
    }

    /* Responsive */
    @media (max-width: 480px) {
        .container {
            width: 90%;
            padding: 25px;
        }

        .input-group {
            flex-direction: column;
        }

        input {
            width: 100%;
        }

        button {
            width: 100%;
        }
    }
</style>
</head>
```




Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
<body>
```

```
<div class="background-animation"></div>
```

```
<div class="container">
```

```
<h1>☁ Weather App</h1>
```

```
<div class="input-group">
```

```
<input type="text" id="city" placeholder="Enter city name">
```

```
<button onclick="getWeather()">Search</button>
```

```
</div>
```

```
<div id="result" class="result-card"></div>
```

```
</div>
```

```
<script>
```

```
function getWeather() {
```

```
let city = document.getElementById("city").value;
```

```
let apiKey = "59bf3ab87498ff51d292c60d8dd2a5c8";
```

```
let url = `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=metric`;
```

```
fetch(url)
```

```
.then(response => response.json())
```

```
.then(data => {
```

```
if (data.cod === 200) {
```

```
let temperature = data.main.temp;
```

```
let humidity = data.main.humidity;
```

```
let condition = data.weather[0].description;
```

```
document.getElementById("result").innerHTML =
```

```
`<p>Temperature: ${temperature} °C</p>
```

```
<p>Humidity: ${humidity}%</p>
```

```
<p>Condition: ${condition}</p>`;
```

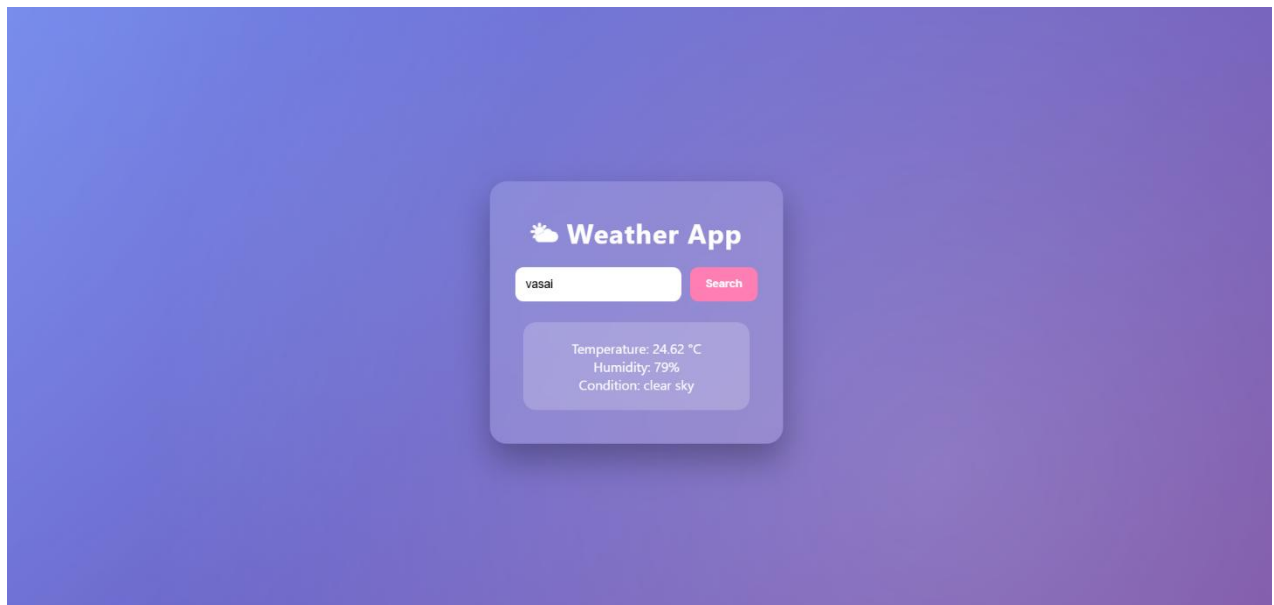


Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
}  
else {  
    document.getElementById("result").innerHTML =  
        "City not found!";  
}  
  
})  
.catch(error => {  
    console.log(error);  
});  
}  
</script>  
</body>  
  
</html>
```

Output:





Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

Conclusion

Thus, a dynamic weather web application was successfully developed using AJAX and a public Weather API. This experiment demonstrates real-time data fetching, asynchronous communication, and dynamic content updating using JavaScript and DOM manipulation

Questions:

1. What is AJAX?

AJAX (Asynchronous JavaScript and XML) is a technique used to send and receive data from a server without reloading the whole web page.

2. What is a Weather API?

A Weather API is a service that provides weather data (like temperature, humidity, forecast) which developers can use in their applications.

3. Why is AJAX preferred over page reload?

AJAX is faster and improves user experience because it updates only part of the page instead of reloading the entire page.

4. Which method is used to fetch API data in JavaScript?

The `fetch()` method is used to get data from an API in JavaScript.